triloop & picoacq

User Manual and Documentation

Three-loop magnetic-field array acquisition and analysis

May 23, 2026

Version 0.1.0

Contents

---

1 Overview

triloop and picoacq are companion software packages for HF radio polarimetry using three orthogonal magnetic-loop antennas:

- picoacq drives a PicoScope 5444D (or any 5000-series) digitiser to capture three-channel coherent recordings, and writes them to a triloop-format HDF5 file.
- triloop reads those files (or any compatible HDF5), extracts the narrow-band complex baseband at a target carrier, recovers the lab-frame magnetic field vector from the three loop measurements, and computes polarization-independent intensity, Stokes parameters, ellipticity, position angle, and a beamformed direction-of-arrival map.

The two packages share a common HDF5 file format (Section 4) so the analysis pipeline is independent of the digitiser used for capture — you can swap in a different acquisition tool by writing the same file format.

1.1 Audience

This manual assumes basic Python familiarity (you can write `import numpy as np` without looking it up), comfort with command-line tools, and a working knowledge of complex baseband signal processing. Mathematical background on the cube-vertex array geometry and polarimetry is covered in the companion technical report *A Three-Loop Magnetic-Field Array for Polarization-Independent HF Reception*; this manual focuses on *using* the software.

1.2 Quick start (five minutes)

The most common workflow is a multi-band capture (all WWV bands at once via bandpass undersampling at 15.625 MS/s) followed by interactive inspection of the resulting HDF5 file in a self-contained HTML browser:

```
# Install
pip install -e ./triloop ./picoacq

# Make a synthetic capture (no hardware required) with all 5 WWV bands
```

```
picoacq capture -o cap.h5 --simulate \
    --sim-pol rcp --sim-faraday 30 --sim-snr 25

# Inspect metadata
triloop summary cap.h5

# Static QC plot (full-Nyquist PSD + per-band zooms + probe table)
triloop view cap.h5

# Interactive HTML browser (pan/zoom + animated polarization ellipse +
# intensity time history with fade CDFs)
triloop browse cap.h5

# Per-band scientific summary across all bands at once
triloop analyze-multi cap.h5 --az 9 --el 35
```

For the older single-band carrier workflow (when you only care about a specific RF tone), use `triloop analyze` or `triloop locate` — those still take `--carrier` explicitly and operate directly on the captured baseband or IF signal.

2 Installation

Both packages are pure Python (with the optional exception of the PicoScope SDK for hardware capture). Recommended Python is 3.10 or later. Install both editable from the project tree:

```
git clone <your-repo-url> triloop-project
cd triloop-project
pip install -e ./triloop          # analysis package
pip install -e ./picoacq          # acquisition package
```

Dependencies

Package	Why
`numpy, scipy`	numerical core
`matplotlib`	figures
`h5py`	HDF5 read/write
`click`	command-line interface
`jupyterlab, ipywidgets`	(optional) for the analysis notebook
`picosdk`	(optional) PicoScope hardware support

For hardware capture you also need to install the PicoSDK system library from https://www.picotech.com/downloads. On macOS this is a `.pkg` installer; on Linux it is a Debian `apt` package or RPM; on Windows it is a regular installer.

Verifying installation

After installation, the following two commands should each print a short help message:

```
triloop --help
picoacq --help
```

3 Coordinate systems and conventions

triloop works in a right-handed Cartesian lab frame (E, N, Up). Azimuth φ is measured *clockwise from North* (so 0° is north, 90° is east). Elevation θ is measured *above the horizontal*. A direction unit vector is

$$\hat{\mathbf{k}}(\varphi, \theta) = \big(\cos\theta \sin\varphi,\ \cos\theta \cos\varphi,\ \sin\theta\big)^{\top}. \tag{1}$$

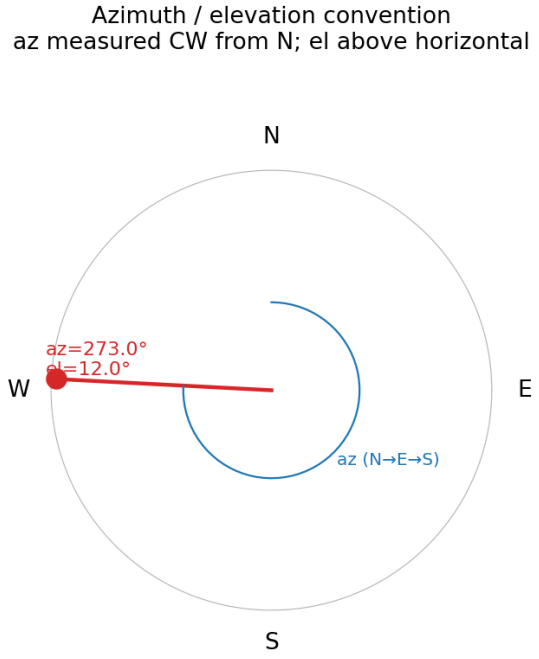


Figure 1: Azimuth/elevation convention used throughout triloop.

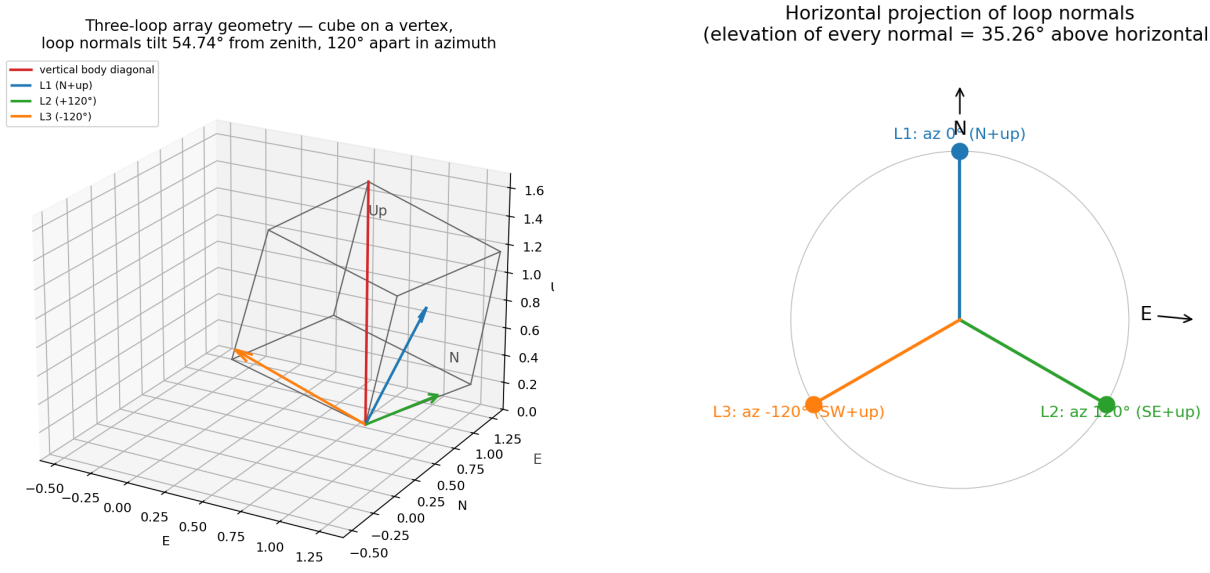
3.1 The default cube-vertex layout

triloop's default geometry assumes the three loops sit on three faces of a cube whose body diagonal is vertical and whose lower vertex is on the ground (Figure 2a). Each loop normal tilts 35.26° above the horizontal; their azimuthal projections are spaced 120° apart. Loop `L1` points N+up, `L2` points +120°, `L3` points −120°.

3.2 Customising the loop geometry

If your array does not match the default, you supply a different `loops_config` dict (in code) or edit it in the notebook. The schema:

```
loops_config = {
    "coordinate_frame":      "ENU",
    "azimuth_convention":    "cw_from_north",
    "elevation_convention":  "above_horizon",
    "loops": [
```



(a) 3-D view.

(b) Top-down view.

Figure 2: Default loop geometry: cube on a vertex with body diagonal vertical. All three normals tilt 35.26° above horizontal.

```
        {"name": "L1", "channel": "A",
         "normal_az_deg":    0.0,     # azimuth of loop normal, deg from N
            CW
         "normal_el_deg":   35.26,    # elevation of loop normal above
            horizontal
         "gain":             1.0,     # per-loop scalar gain calibration
         "phase_offset_deg": 0.0,     # per-loop electrical-phase
            calibration
        },
        {"name": "L2", "channel": "B", "normal_az_deg": 120.0, ...},
        {"name": "L3", "channel": "C", "normal_az_deg": -120.0, ...},
    ],
}
```

The `channel` field must match the channel names in the HDF5 capture (e.g. `"A"`, `"B"`, `"C"` for PicoScope ports A, B, C). Per-loop `gain` multiplies the corresponding row of the geometry matrix N; `phase_offset_deg` is applied to the complex baseband after extraction.

4 File format

Both picoacq and triloop use the same HDF5 v0.1 file format:

```
/                       HDF5 root, attribute @format_version = "0.1"
/metadata               group, with attributes:
    @sample_rate        float64, Hz
    @start_time_utc     ISO-8601 string
    @duration_s         float64
    @scope_model        string
    @scope_serial       string
```

```
    @capture_settings       JSON-encoded dict
    @loops_config           JSON-encoded dict
/channels                   group
    /A   ...                int16 or float32, length N
    /B
    /C
    /D                      optional reference channel
    Each channel dataset has attributes:
        @gain_v_per_count   float64
        @offset_v           float64
/time                       optional float64 dataset, length N
                            (regenerated from sample_rate if absent)
```

triloop.io_hdf5 provides `read_capture()` and `write_capture()` which any third party can use to interoperate with the toolchain.

5 The `picoacq` tool

5.1 PicoScope 5444D hardware

picoacq is written for the Pico Technology PicoScope 5444D ("MSO," the variant with digital-channel breakout connectors, or the non-MSO variants 5444B and 5444A which differ only in form factor). Key properties relevant to coherent multi-channel acquisition:

- **4 analog input channels** (BNC, switchable $1\,\mathrm{M\Omega} \,\|\, 15\,\mathrm{pF}$ *or* $50\,\Omega$).
- **Variable-resolution "FlexRes" ADC** that operates from 8 bits through 16 bits, with a corresponding sample-rate trade-off. For HF work in the operating mode triloop uses, the relevant point is **16-bit mode at 62.5 MS/s on all four channels simultaneously**.
- **Synchronous, time-aligned 4-channel sampling.** All four channels share a single sample clock (one PLL) and a single trigger; channel-to-channel sample skew within a single capture is sub-picosecond (i.e. insignificant compared with any HF observable of interest).
- **ADC architecture.** The 5444D's FlexRes architecture can trade off bit-depth against per-channel sample rate, with the highest resolutions (15- and 16-bit) achieved by combining ADC cores via interleaving or pairing. In the 16-bit four-channel mode that triloop uses, all four channels are produced as time-aligned sample streams; users requiring a definitive chip-level block diagram should consult the manufacturer's datasheet. What matters for polarimetric work is that Pico specifies channel-to-channel sample skew below one sample period (i.e. $< 16\,\mathrm{ns}$ at $62.5\,\mathrm{MS/s}$) and that this skew, plus any per-channel phase response of the analog front-end, is factory-calibrated and stored on the unit.
- **Reference clock** can be switched between the internal PLL and an external 10 MHz input, enabling phase-locked operation against a GPS-disciplined oscillator if required.
- **Streaming over USB 3.0** sustainably at $\sim 400\,\mathrm{MB/s}$, enabling continuous capture at $62.5\,\mathrm{MS/s} \times 4$ channels $\times$ 14-bit-packed.

Coherence implication: the four channels of a 5444D are sampled at the *same instant* (within picoseconds), so the channel-to-channel phase relations the analysis pipeline relies on are preserved exactly. Three of the four channels carry the loop signals (`A`, `B`, `C`); the fourth (`D`) is available for an optional reference such as a 1 PPS GPS pulse, a stable oscillator beacon, or simply unused.

5.2 Software requirements

To drive the scope from picoacq you need two things:

1. The **PicoSDK system installer** from https://www.picotech.com/downloads. This installs the shared library (`libps5000a.dylib` on macOS, `libps5000a.so` on Linux, `ps5000a.dll` on Windows) used to talk to the device.
2. The `picosdk` Python wrapper: `pip install picosdk` (or `pip install -e ./picoacq[hardware]`).

picoacq accesses the PicoSDK via `ctypes`, configures the scope into 16-bit four-channel block-acquisition mode, requests the user's sample rate (selected by Pico's discrete *timebase* table; the closest available rate at or below the requested value is used), and arms a software trigger. After the block completes the raw `int16` samples are pulled to host memory and converted to volts using the per-channel range setting.

If the scope is not present or the SDK is missing, picoacq prints a one-line warning and silently substitutes the simulator described in Section 5.6 so the rest of the analysis toolchain can be tested without hardware.

5.3 Field setup with PicoScope 7 (the GUI)

Before driving the scope from picoacq, it is almost always worth confirming the hardware works using Pico's free GUI application **PicoScope 7** (https://www.picotech.com/downloads). The macOS download is a regular `.dmg`; drag `PicoScope.app` to `/Applications`. The bundled user-space USB plugin is authorised on first launch.

PicoScope 7 is the right tool for:

- **First-light sanity check.** Plug in the USB cable, launch PicoScope 7, enable channels A/B/C, and confirm you see live traces from each loop preamp. This rules out cabling and driver issues before any code runs.
- **Choosing voltage ranges.** Watch the live time-domain trace to set each channel's ±range so the strongest expected signal uses most of the dynamic range without clipping. Pass those values to `picoacq capture --range` *V*.
- **Live FFT.** PicoScope 7 has a built-in spectrum view. Confirm that the WWV or CHU carrier you are about to record is actually present (and at the expected detuned offset, e.g. near 2 kHz) before committing to a 60 s capture.
- **Noise floor.** In spectrum view with the input terminated or the antenna disconnected, you can quickly verify your loop preamps aren't injecting birdies, oscillations, or excess broadband noise.
- **Trigger setup.** Edge / threshold / window triggers are easier to dial in interactively than via the SDK. Once a useful configuration is found, replicate it in your picoacq command-line invocation.

PicoScope 7 holds an exclusive handle on the device while running, so **quit PicoScope 7 before invoking picoacq capture**, or the SDK call will fail with a "device in use" error. Both applications use the same factory calibration, so the channel-to-channel phase relations measured in PicoScope 7 carry over to picoacq captures.

Recommended workflow.

1. Install `PicoScope.app` from Pico's downloads page.
2. Plug the scope in via USB-3. Authorise the kernel/user-space plugin on first launch when macOS prompts.
3. Connect loop preamps to channels A, B, C.
4. Use PicoScope 7 to set the voltage ranges and verify signal presence; record those ranges.
5. Quit PicoScope 7.
6. Install the PicoSDK system installer and the `picosdk` Python wrapper (Section 5.1).
7. Run `picoacq capture` from the command line with the ranges you chose.

5.4 One-shot captures

```
# Default workflow: 8 s capture of all 5 WWV bands at once via
# bandpass undersampling at 15.625 MS/s, auto-ranged.
picoacq capture -o cap.h5             # all defaults

# Single-band IF capture (e.g. classical 200 kSPS audio-IF reception)
picoacq capture -o cap.h5 \
    --duration 60 --rate 200000 \
    --channels A,B,C --range 2.0 --no-auto-range \
    --rf-bands ""                     # disable rf_bands metadata
```

The default rate and `--rf-bands` setting in picoacq are tuned for multi-band undersampling; see the *picoacq User Manual* for the band-to-baseband alias map and onboard-memory limits.

If the PicoSDK is not available or the scope is not connected, picoacq prints a warning and falls back to the simulator described in Section 5.6.

To force the simulator (e.g. on a laptop without hardware), pass `--simulate` along with the simulation knobs:

```
picoacq capture -o cap.h5 --simulate \
    --sim-pol     rcp \                 # or linear_vertical, lcp, ...
    --sim-faraday 30 \                  # deg/s Faraday rotation rate
    --sim-snr     25 \                  # per-channel SNR (dB)
    --sim-az      273 \
    --sim-el      12  \
    --sim-carrier 25000                 # IF carrier freq (Hz)
```

5.5 Continuous monitoring

For long-duration unattended runs, use `picoacq monitor`. It takes one capture every `--interval` seconds and writes a timestamped file:

```
picoacq monitor -d /data/triloop_runs/ \
    --interval 600 \                    # seconds between capture starts
    --duration 8 \                      # length of each capture (default)
    --prefix   wwv
# Stop with Ctrl-C, or pass --n-captures N to bound it.
```

The output filenames look like `wwv_20260520T144312Z.h5`. SIGTERM and SIGINT are handled cleanly — the current capture finishes and the loop exits.

5.6 The simulator

The simulator generates a synthetic three-loop dataset with controllable arrival direction, polarization, additive Gaussian noise, and Faraday rotation rate. It is used both:

1. by `picoacq capture --simulate` (and as the no-hardware fallback) to write a fully-formed HDF5 file with simulated data;
2. by `triloop simulate` to do the same thing without any acquisition driver in the loop.

The polarization options are `linear_vertical, linear_horizontal, rcp, lcp, elliptical`. Faraday rotation rotates the polarization ellipse in the plane perpendicular to $\hat{\mathbf{k}}$ at the specified rate; for linearly-polarized input this produces classical Faraday fading on a linear-pol receiver, while for circularly-polarized input it produces no amplitude effect (just a common phase advance) — a useful sanity check.

6 The `triloop` tool

6.1 Inspecting a capture

```
triloop summary cap.h5
```

prints something like:

```
file:           cap.h5
format_version: 0.1
start_time_utc: 2026-05-20T14:43:12.345678+00:00
sample_rate:    200000 Hz
duration_s:     60 s
scope:          PicoScope 5444D (s/n EW123/4567)
channels:
  A: n=12000000, range=[-1.853, +1.967]
  B: n=12000000, range=[-1.732, +1.811]
  C: n=12000000, range=[-1.624, +1.703]
loops_config: 3 loops
```

6.2 Single-file analysis

```
triloop analyze cap.h5 \
    --carrier 25000 \           # target carrier (Hz)
    --bw      2000 \            # analysis bandwidth (Hz, +/- BW/2 retained)
    --az      273 \             # initial guess azimuth (deg from N CW)
    --el      12  \             # initial guess elevation (deg above horizon)
    [--out summary.json]
```

The output is a JSON object on stdout (or written to `--out`):

```
{
  "input_file":              "/path/to/cap.h5",
  "sample_rate_Hz":          200000.0,
  "duration_s":              60.0,
  "f_peak_Hz":               25000.0954,
  "snr_db_per_loop":         [76.4, 76.5, 76.4],
  "az_initial_deg":          273.0,
```

```
  "el_initial_deg":          12.0,
  "median_pol_fraction":     1.0,
  "median_ellipticity_deg": -44.76,
  "median_position_angle_deg": 0.87,
  "instant_freq_mean_Hz":    25000.083,
  "intensity_mean":          0.99996,
  "intensity_std":           0.0109
}
```

6.3 Direction finding (`triloop locate`)

The `analyze` command requires the user to supply an initial (φ, θ) guess, which it then uses for the perp-plane projection. When the source direction is unknown, or when you want the *sharpest possible* direction estimate, use `triloop locate` instead:

```
triloop locate cap.h5 \
    --carrier 25000 --bw 2000 \
    --az0 270 --el0 15 \                # initial guess for the null sweep
    --half-width 30 --n-points 81 \     # local sweep grid
    --out-png residual_map.png          # optional residual heat-map
```

The pipeline is:

1. Extract per-loop complex baseband at the carrier (same as `analyze`).
2. Form the lab-frame 3-vector $\mathbf{z}_{\text{lab}}(t) = \mathsf{N}^{-1}\mathbf{z}_{\text{loops}}(t)$ and compute the 3×3 coherency matrix $\mathsf{C} = \langle \mathbf{z}_{\text{lab}} \mathbf{z}_{\text{lab}}^{H} \rangle$.
3. **Eigendecomposition.** The smallest-eigenvalue eigenvector of C is $\pm\hat{\mathbf{k}}_{\text{true}}$ (modulo the front/back ambiguity and the polarization degeneracy described in Section 9.5). This gives the *coarse* direction estimate.
4. **Null sweep.** A 2-D grid of trial directions around that estimate (or around the user's `--az0/--el0`, if supplied) is searched to minimise the perp-residual energy $\langle |\hat{\mathbf{k}}_{\text{trial}} \cdot \mathbf{z}_{\text{lab}}|^2 \rangle$. A parabolic fit on the 3×3 neighbourhood of the grid minimum yields a sub-grid direction.
5. The locked ($\hat{\mathbf{k}}_{\text{locked}}$) is then used to project, recover the polarization basis, and compute the Stokes parameters and intensity (same as `analyze`).

Why this works better than maximizing the directional response: finding a maximum places you on a shallow plateau of $\cos^2 \beta$, while finding a null places you on the steep slope of $\sin^2 \beta \sim \beta^2$. In typical conditions the null can be located $\sim 10\times$ more precisely than the response maximum, with the residual sometimes 40 dB or more below its peak.

The Python API mirrors the CLI:

```
from triloop import lock_and_analyze, read_capture
cap = read_capture("cap.h5")
res = lock_and_analyze(cap["time"],
                       cap["channels"]["A"], cap["channels"]["B"],
                       cap["channels"]["C"],
                       f0=25e3, BW=2e3,
                       az0=270, el0=15,
                       half_width_deg=30, n_points=81)
print(f"locked direction: az={res['az_locked']:.3f}  , "
```

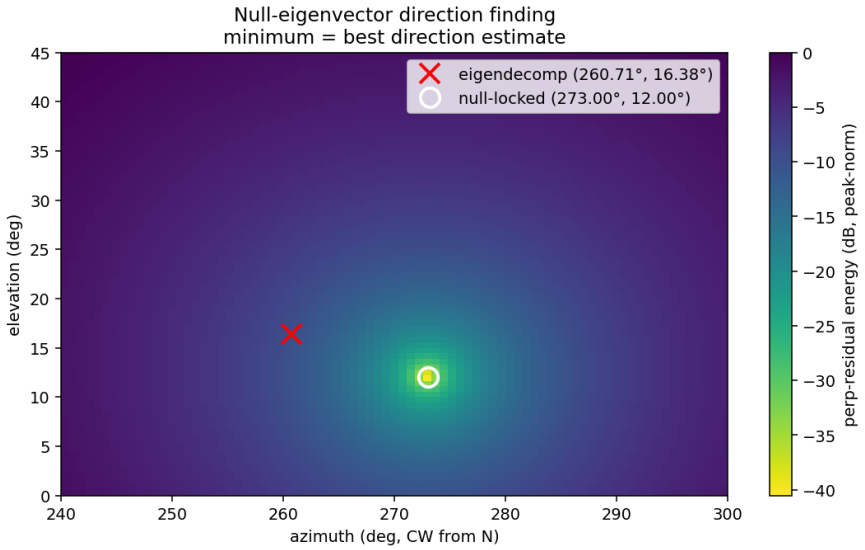


Figure 3: Output of `triloop locate` on a high-SNR RCP synthetic capture. The eigendecomp seed (red x) was displaced $\sim 12°$ in azimuth from truth (273°, 12°); the null sweep refined it to $(272.998°, 12.002°)$, sub-0.01° accuracy. Residual depth at the locked direction is 40 dB below the off-axis maximum — this is the "software null" equivalent of rotating a direction-finding loop by hand.

```
      f"el={res['el_locked']:.3f}  ")
print(f"null strength:   {res['eig']['null_strength']:.2e}")
```

When NOT to use `locate`. For purely linearly polarized signals from a single station, the rank-1 coherency leaves the direction ambiguous along an entire plane (Section 9.5). `triloop locate` will return *a* direction — the one with the smallest residual on its grid — but that direction has no physical meaning. Use one of the rank-2-restoring conditions (circular pol, Faraday rotation, multi-station triangulation) for trustworthy localisation.

6.4 Batch analysis

```
triloop batch /path/to/captures/ \
    --carrier 25000 --bw 2000 --az 273 --el 12 \
    --pattern '*.h5' \
    [--workers 4] \
    [--out triloop_batch_summary.jsonl]
```

writes one JSON line per capture into a single `.jsonl` file (default: `triloop_batch_summary.jsonl` alongside the inputs). This loads cleanly into pandas:

```
import pandas as pd
df = pd.read_json("triloop_batch_summary.jsonl", lines=True)
df["t"] = pd.to_datetime(df["start_time_utc"])
df.plot(x="t", y="median_ellipticity_deg")
```

6.5 Multi-band analysis (`triloop analyze-multi`)

When the capture file was produced by `picoacq` at the default 15.625 MS/s rate, every WWV band (2.5, 5, 10, 15, 20 MHz) is present in a single recording at a different baseband alias position (see the *picoacq* manual, §Default rate). `analyze-multi` runs the full polarimetry pipeline once per band, in a single command, and produces both a JSON-per-band summary and a 5×4 comparison plot:

```
triloop analyze-multi cap.h5 --az 9 --el 35 \
    [--bw 4000] [--decim 20000] \
    [--rf-bands "2.5e6,5e6,10e6,15e6,20e6"] \
    [--loops-channels A,B,C] \
    [--out-png cap_multiband.png] \
    [--out-json cap_multiband.json]
```

The list of RF bands is read from the file's `capture_settings/rf_bands_hz` field by default; pass `--rf-bands` only to override or restrict the set. Each band's baseband location is computed via `picoacq.alias.alias_of(rf, sample_rate)`; even-Nyquist-zone bands are conjugated so their analysis spectra read in the correct frequency sense (see the *picoacq* manual).

For each band the pipeline does:

1. Brick-wall mix-and-LPF the real signal of every selected channel down to a complex baseband centred on the band's alias frequency, keeping ±`bw`/2 of bandwidth.
2. Decimate to roughly `decim_rate_hz` (default 20 kHz), which keeps phase information intact at the carrier scale of interest while shrinking arrays by 3 orders of magnitude.
3. Run `analyze_z_loops()` on the resulting (3, N) complex baseband matrix, exactly as `analyze` would on directly-mixed data.

The output JSON has one record per band:

```
{
  "input_file": "...",
  "az_deg": 9.0, "el_deg": 35.0,
  "bw_hz": 4000.0, "decim_rate_hz": 20000.0,
  "loops_channels": ["A", "B", "C"],
  "bands": [
    {"rf_hz": 2500000.0, "baseband_hz": 2500000.0,
     "nyquist_zone": 1, "inverted": false,
     "snr_db_per_loop": [88, 89, 89],
     "median_pol_fraction": 1.0,
     "median_ellipticity_deg": 0.1,
     "median_position_angle_deg": 0.4,
     "intensity_mean": 1.0, "intensity_std": 0.02,
     "instant_freq_mean_Hz": 2500000.0},
    ...
  ]
}
```

Each band's row in the comparison PNG shows: intensity time series, instantaneous-frequency residual, ellipticity (deg), and polarization fraction.

6.6 Quick-look QC (`triloop view`)

`triloop view` produces a single static PNG for fast eyeballing:

```
triloop view cap.h5 [--out-png cap_view.png] [--bw 8000]
```

Layout (top to bottom): full-Nyquist PSD with all channels overlaid and each RF band annotated at its baseband alias position; per-band PSD zooms (axis flipped for inverted/even-zone bands so frequencies read correctly); a decimated time-domain trace of all channels; and the auto-range probe table from `capture_settings/auto_range_probe` (peak / RMS / chosen PicoScope range per channel). This is the right tool to confirm "did the capture work?" before doing any heavier analysis.

6.7 Interactive HTML browser (`triloop browse`)

`triloop browse` produces a self-contained HTML file (no server, no external network — embeds Plotly via CDN) that lets you pan, zoom, hover, and animate across the capture interactively:

```
triloop browse cap.h5 \
    [--out-html cap_browse.html] \
    [--az 9 --el 35] \
    [--bw 4000] [--decim 20000] \
    [--n-anim-frames 60] \
    [--open-browser]
```

Layout: a tab bar at the top, with one "Overview" tab containing the full-Nyquist PSD (channels toggle-able via the legend, RF-band markers as dashed verticals), one "Time series" tab with decimated time-domain traces, and one tab *per RF band* containing four panels:

1. **PSD zoom** around the band centre (axis flipped for inverted bands).
2. **Intensity time history** with linear $|\mathbf{B}_\perp|^2$ on the primary axis and dB-relative-median on a togglable secondary axis — useful for picking out fades.
3. **Intensity CDF** with P10 (10th-percentile) and P1 (1st-percentile) fade-depth markers, the classical fade-margin metric for HF link-budget work.
4. **Animated polarization ellipse** drawn in the $(\mathrm{Re}\, A_p, \mathrm{Re}\, A_q)$ plane, with a Play/Pause control and a frame slider, so Faraday rotation and time-varying ellipticity become directly visible.

The page is a single `.html` file you can email, version, scp, or open in any modern browser. No Python or server needed at viewing time.

6.8 Interactive analysis (Jupyter notebook)

```
triloop notebook cap.h5
```

opens the bundled analysis notebook with the file path pre-loaded. The notebook has seven cells:

1. **Load capture** — reads metadata, prints summary.
2. **Loop geometry config** — edit this cell to match your array (Section 3.2).
3. **Analysis parameters** — carrier, bandwidth, initial direction guess.

4. **Raw signals and spectra** — 20 ms time-domain plot plus FFT power-spectra on all three loops.
5. **Run the pipeline** — calls `analyze()` and prints summary statistics.
6. **Time-resolved diagnostics** — amplitude, phase residual, instantaneous frequency, polarization observables.
7. **Beamforming sweep** — heat-map of $P(\varphi, \theta)$ with the initial guess and best-fit marked.

7 Worked examples

7.1 Example 1: linearly polarized signal with Faraday rotation

A linearly-polarized 25 kHz IF carrier with 30°/s Faraday rotation, 25 dB per-channel SNR, arriving from $\varphi = 273°$, $\theta = 12°$ (the WWV → Cambridge geometry):

```
triloop simulate cap.h5 --pol linear_vertical \
    --faraday 30 --snr 25 --duration 4
triloop analyze cap.h5 --carrier 25000 --bw 2000 --az 273 --el 12
```

The recovered `f_peak` matches truth within tens of mHz; the median polarization fraction is ≈ 1.0 (fully polarized); the median ellipticity is near 0° (linear); the position angle sweeps the expected 30°/s. Figures 4–6 show the per-cell output of the analysis notebook.

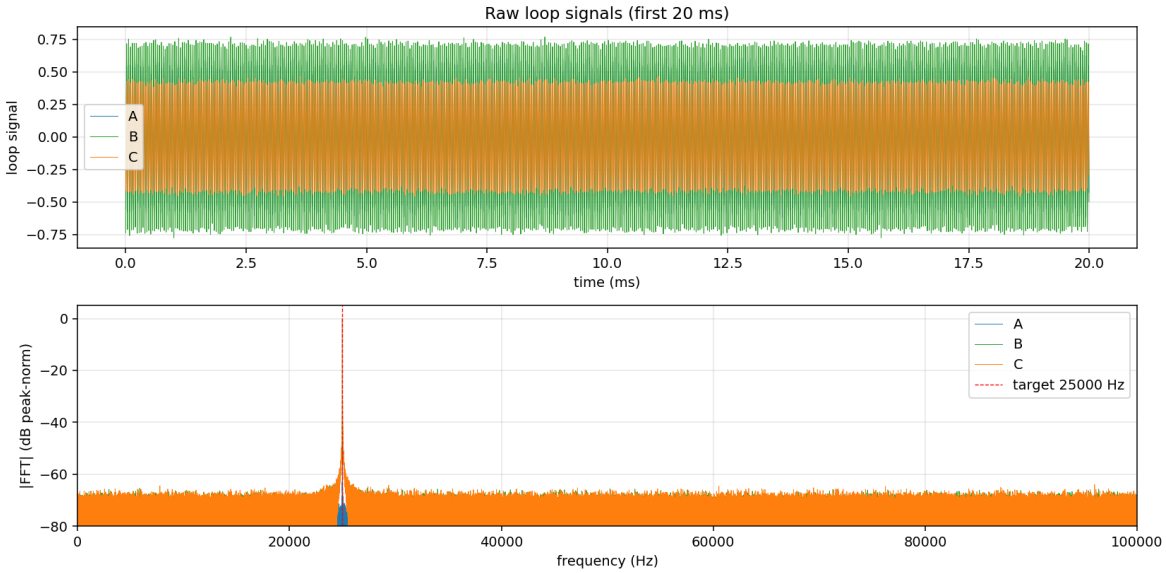


Figure 4: Raw three-loop signals (top) and per-loop FFT power spectra (bottom) for the linear-pol simulated capture. The dashed red line marks the requested carrier (25 kHz); the spectral peak is clearly visible above the noise floor on all three loops, with channel-to-channel amplitude ratios consistent with the cube-vertex projection of a single linear polarization.

7.2 Example 2: right-circular polarization (the Faraday-immune control)

For *circularly* polarized input, Faraday rotation only adds a common phase — the wave's amplitude is unaffected. A useful sanity check on the pipeline:

```
triloop simulate cap_rcp.h5 --pol rcp --faraday 30 --snr 25 --duration 4
triloop analyze cap_rcp.h5 --carrier 25000 --bw 2000 --az 273 --el 12
```

The analysis returns ellipticity $\approx -45°$ (RCP) and *constant* amplitude in both the polarization-independent and dominant-pol traces.

8 Python API reference

8.1 Top-level functions

```
from triloop import (
    read_capture, write_capture,
    analyze, analyze_z_loops, AnalysisResult,
    extract_bands, BandExtraction,
    analyze_all_bands, MultiBandResult, make_multiband_figure,
    make_view_figure,
    beamform_grid,
    estimate_direction_from_z_lab,
    null_search, perp_residual_energy, parabolic_refine,
    lock_and_analyze,
    default_loops_config,
    az_el_to_khat, perp_projector, perp_orthonormal_basis,
    extract_complex_baseband,
    compute_stokes,
)
# (the interactive browser builder lives in a submodule:)
from triloop.browse import build_browser_html, write_browser
```

`read_capture(path) -> dict`
Read a triloop HDF5 file. Returns a dict with keys `sample_rate`, `duration_s`, `start_time_utc`, `scope_model`, `scope_serial`, `capture_settings`, `loops_config`, `channels`, `channel_raw`, `time`, `format_version`.

`write_capture(path, channels, sample_rate, ...)`
Write a triloop HDF5 file. `channels` is a dict mapping channel name to 1-D numpy array. Optional kwargs include `loops_config`, `capture_settings`, `scope_model`, `scope_serial`, `channel_gains`, `channel_offsets`, `store_time_array` (default `False`: regenerate t from rate), `compress` (default `True`: gzip level 4).

`analyze(t, B1, B2, B3, f0, BW, az_deg, el_deg, loops_config=None) -> AnalysisResult`

Run the full pipeline. Returns a dataclass with attributes: `f_peak`, `snr_db_per_loop`, `z_loops`, `z_lab`, `z_perp`, `intensity`, `A_p`, `A_q`, `stokes_I`, `stokes_Q`, `stokes_U`, `stokes_V`, `pol_fraction`, `ellipticity_deg`, `position_angle_deg`, `amp_dominant`, `instant_phase`, `instant_freq`, `instant_freq_mean`, `khat`, `p_hat`, `q_hat`, `loops_config`, `sample_rate`, `duration_s`.

`analyze_z_loops(t, Z_loops, f_peak, snrs_db, az_deg, el_deg, loops_config=None) -> AnalysisResult`

Same pipeline as `analyze`, but operates on a pre-extracted complex-baseband array `Z_loops` of shape $(3, N)$ instead of real-valued loop signals. Used by the multi-band path so the FFT-based mix/LPF only happens once per band rather than once per call.

`extract_bands(cap, rf_bands=None, bw_hz=4000.0, decim_rate_hz=20000.0, channels=None) -> dict`
Multi-band complex-baseband extractor for bandpass-undersampled captures. For every `rf_bands` entry (defaults to `cap['capture_settings']['rf_bands_hz']`) the function reads the file's sample rate, computes the band's baseband alias position via `picoacq.alias.alias_of`, mixes-and-LPFs every selected channel down, decimates to `decim_rate_hz`, and conjugates even-Nyquist-zone slices so the analysis sees the spectrum the right way around. Returns a dict mapping `rf_hz` (float) to a `BandExtraction` dataclass with fields: `rf_hz, baseband_hz, nyquist_zone, inverted, bw_hz, decim_rate_hz, t, z` (where `z` is a dict mapping channel name to `complex64` array).

`analyze_all_bands(cap, az_deg, el_deg, loops_channels=("A","B","C"), bw_hz=4000.0, decim_rate_hz=20000.0, rf_bands=None, loops_config=None) -> dict`
Convenience wrapper that calls `extract_bands` and then `analyze_z_loops` on each band. Returns a dict mapping `rf_hz` to a `MultiBandResult` (`rf_hz, extraction, analysis, snrs_db`).

`make_multiband_figure(results, out_path=None)`
Build a Matplotlib figure with one row per band × four columns (intensity, instantaneous frequency, ellipticity, polarization fraction). Either save to `out_path` or return the figure object.

`make_view_figure(cap, rf_bands=None, bw_hz=8000.0, out_path=None)`
Build the static QC PNG used by the `view` subcommand.

`build_browser_html(cap, az_deg=9, el_deg=35, loops_channels=("A","B","C"), bw_hz=4000, decim_rate_hz=20000, n_anim_frames=60, title=None) -> str`
Return an HTML string containing the full interactive browser (**Section 6.7**). The companion `write_browser(cap, out_path, **kw)` writes it to disk.

`beamform_grid(z_loops, loops_config, az_grid_deg=None, el_grid_deg=None) -> (P, az, el)`

Compute the time-averaged $P(\varphi, \theta)$ on a 2-D grid.

`extract_complex_baseband(t, b, f0, BW) -> (z, f_peak, snr_db)`
Find the spectral peak nearest to `f0` within $\pm$`BW` in the real signal `b(t)`, mix to baseband, and brick-wall low-pass to $\pm$`BW/2`.

`estimate_direction_from_z_lab(Z_lab) -> dict`
Eigendecompose the lab-frame coherency matrix $\mathsf{C} = \langle \mathbf{z}\mathbf{z}^H \rangle$ and return the smallest-eigenvalue eigenvector as $\hat{\mathbf{k}}_{\text{est}}$, its (φ, θ), the eigenvalues (descending), the polarization basis ($\hat{\mathbf{p}}$, $\hat{\mathbf{q}}$ from the two largest eigenvectors), and a `null_strength` λ_3/λ_1.

`null_search(Z_lab, az0, el0, half_width_deg, n_points) -> dict`
2-D grid sweep around an initial guess that minimises $\langle|\hat{\mathbf{k}}_{\text{trial}} \cdot \mathbf{z}_{\text{lab}}|^2\rangle$. Returns the residual map and best-on-grid direction.

`lock_and_analyze(t, B1, B2, B3, f0, BW, az0=None, el0=None, ...) -> dict`
End-to-end direction-finding plus polarimetry: extract complex baseband, eigendecompose, run the null sweep (optional), parabolic sub-grid refinement, then perp-plane projection and Stokes computation locked to the recovered direction. See Section 6.3.

8.2 Example: scripted analysis

```
import numpy as np
from triloop import read_capture, analyze, beamform_grid

cap = read_capture("cap.h5")
B1, B2, B3 = cap["channels"]["A"], cap["channels"]["B"], cap["channels"]["
    C"]
res = analyze(cap["time"], B1, B2, B3,
              f0=25_000.0, BW=2000.0, az_deg=273.0, el_deg=12.0,
              loops_config=cap["loops_config"])

print(f"f_peak = {res.f_peak:.4f} Hz")
print(f"polarization fraction = {np.median(res.pol_fraction):.3f}")
print(f"ellipticity (deg) = {np.median(res.ellipticity_deg):+.2f}")

# beamform
P, az, el = beamform_grid(res.z_loops, cap["loops_config"])
i, j = np.unravel_index(np.argmax(P), P.shape)
print(f"best az,el = ({az[j]:.0f}, {el[i]:.0f})")
```

9 Calibration and known limitations

9.1 Per-loop calibration is critical

The analysis assumes the three loops have equal effective area, gain, and phase response at the operating frequency. Real loops never quite do. Without calibration, intensity estimates can be off by a few %, position angles by a few degrees, and ellipticity may be biased away from 0° even for nominally linear input.

The cleanest calibration procedure:

1. Transmit a known-direction CW signal of known polarization (e.g. a vertically polarized test transmitter at a known site).
2. Capture with `picoacq capture`.
3. Compare the analysis output's recovered position angle and ellipticity to ground truth.
4. Solve for per-loop `gain` and `phase_offset_deg` that drive the recovered values to ground truth, and store them in the `loops_config` for production use.

9.2 Front/back ambiguity

A single-point three-loop B-field array is symmetric under $\hat{\mathbf{k}} \to -\hat{\mathbf{k}}$: the beamforming map satisfies $P(\varphi, \theta) = P(\varphi + 180°, -\theta)$. This is intrinsic geometry, not a software bug; resolving it requires either an additional E-field probe or a phased pair of triplets.

9.3 Coherent sampling

triloop requires that the three loops be sampled with a shared LO and ADC clock — i.e. a single multi-channel digitiser like the PicoScope 5444D, NOT three independent KiwiSDRs. Without coherent sampling, channel-to-channel phase information is corrupted by independent oscillator drift, and Stokes parameters become unrecoverable.

9.4 SNR gain over a naïve power sum

For an *isotropic noise* background and the cube-vertex geometry, the coherent direction-formed estimator $|\mathbf{B}_\perp(\hat{\mathbf{k}})|^2$ achieves an SNR exactly 3/2 (+1.76 dB) higher than incoherently summing the three loop powers $\sum_i |B_i|^2$, regardless of the source direction. Two ingredients give this result: the cube-vertex loop normals form a tight frame ($\sum_i \hat{\mathbf{n}}_i \hat{\mathbf{n}}_i^\top = \mathsf{I}$), which makes the loop-frame noise transformation to lab frame exactly isotropic; and projecting onto the plane perpendicular to $\hat{\mathbf{k}}$ removes one of three orthogonal noise dimensions while preserving the full signal (which lives in that perpendicular plane by construction). The gain is +1.76 dB at every (φ, θ).

9.5 Single-point direction finding is impossible for purely linearly polarized signals

A single linearly-polarized plane wave produces a *rank-1* coherency matrix $\mathsf{C} = \langle \mathbf{z}_{\text{lab}} \mathbf{z}_{\text{lab}}^H \rangle$ — only one eigenvalue is non-zero, and the corresponding eigenvector points along the polarization, not along $\hat{\mathbf{k}}$. The two remaining eigenvalues are degenerate at zero (in the noise-free limit), so the smallest-eigenvalue eigenvector — which we use to estimate $\hat{\mathbf{k}}$ — is ambiguous in any direction within the 2-D plane perpendicular to $\hat{\mathbf{e}}_{\text{pol}}$.

Equivalently: the perp-residual energy $\langle |\hat{\mathbf{k}}_{\text{trial}} \cdot \mathbf{B}(t)|^2 \rangle$ is zero whenever $\hat{\mathbf{k}}_{\text{trial}} \perp \hat{\mathbf{e}}_{\text{pol}}$, not just when $\hat{\mathbf{k}}_{\text{trial}} = \pm \hat{\mathbf{k}}_{\text{true}}$. The null is a plane, not a point.

To direction-find a linearly polarized signal from a single 3-loop station you must break the rank-1 condition by introducing a second linearly-independent polarization sample. Three options work in practice:

- **Faraday rotation:** if the propagation path imprints a time-varying polarization rotation, the polarization axis sweeps through a range of angles during the integration time and the time-averaged coherency becomes rank-2. At HF this happens naturally on most ionospheric paths, especially at lower frequencies where Faraday is large.
- **Circularly polarized source.** RCP and LCP are intrinsically rank-2: the perp-plane is filled both by $\hat{\mathbf{p}}$ and $\hat{\mathbf{q}}$. The null eigenvector is unambiguous.
- **Triangulation.** Two spatially separated three-loop stations resolve the ambiguity, because the rank-1 plane is different at each station and they intersect along $\hat{\mathbf{k}}_{\text{true}}$.

The eigendecomposition reports a small but *non*-zero `null_strength` (λ_3/λ_1) when the rank-1 condition is broken in any of these ways. When `null_strength` is small, the direction estimate

is trustworthy. When it approaches the ratio of the second-smallest to largest eigenvalue (i.e. the rank-1 nullspace is degenerate), trust the result only after polarization diversity has been confirmed.

9.6 Beam pattern is broad

The polarization-independent estimator $|\mathbf{B}_\perp|^2$ has a $\frac{1}{2} + \frac{1}{2}\cos^2\beta$ angular pattern relative to $\hat{\mathbf{k}}_{\text{target}}$. This is a *factor of two* between on-axis and broadside, not a sharp pencil beam. Direction finding via beamforming is therefore approximate; for the cleanest direction estimate, eigendecompose $\langle z_i z_j^* \rangle$ and read off the smallest-eigenvalue eigenvector — this is implemented in upcoming versions.

10 Appendix: file layout in the source tree

```
triloop/                            analysis package
  pyproject.toml
  README.md
  triloop/
    __init__.py
    geometry.py         -- N matrix, projector, p-hat / q-hat basis
    extract.py          -- narrow-band complex baseband (single tone)
    bands.py            -- multi-band extraction (calls picoacq.alias)
    stokes.py           -- Stokes parameters from (A_p, A_q)
    analyze.py          -- analyze() + analyze_z_loops() + AnalysisResult
    multiband.py        -- analyze_all_bands(), make_multiband_figure()
    direction.py        -- null-eigenvector + null-sweep direction finding
    beamform.py         -- P(az, el) sweep
    view.py             -- static QC PNG (matplotlib)
    browse.py           -- interactive HTML browser (Plotly)
    io_hdf5.py          -- HDF5 reader / writer
    config.py           -- loops_config dict + validator
    cli.py              -- click-based command-line interface
  notebooks/
    triloop_analysis.ipynb
  tests/
    test_simulated_recovery.py
  docs/
    triloop_user_manual.tex      (this file)
    triloop_analysis_ref.tex     (developer's reference)
    figures/

picoacq/                            acquisition package
  pyproject.toml
  README.md
  picoacq/
    __init__.py
    ps5444a.py          -- thin PicoSDK 5000a wrapper
    simulator.py        -- no-hardware fallback
    recorder.py         -- capture orchestrator + onboard-buffer guard
    alias.py            -- bandpass-undersampling alias map (RF<->baseband)
    cli.py              -- click-based CLI (capture + monitor)
  docs/
    picoacq_user_manual.tex
```

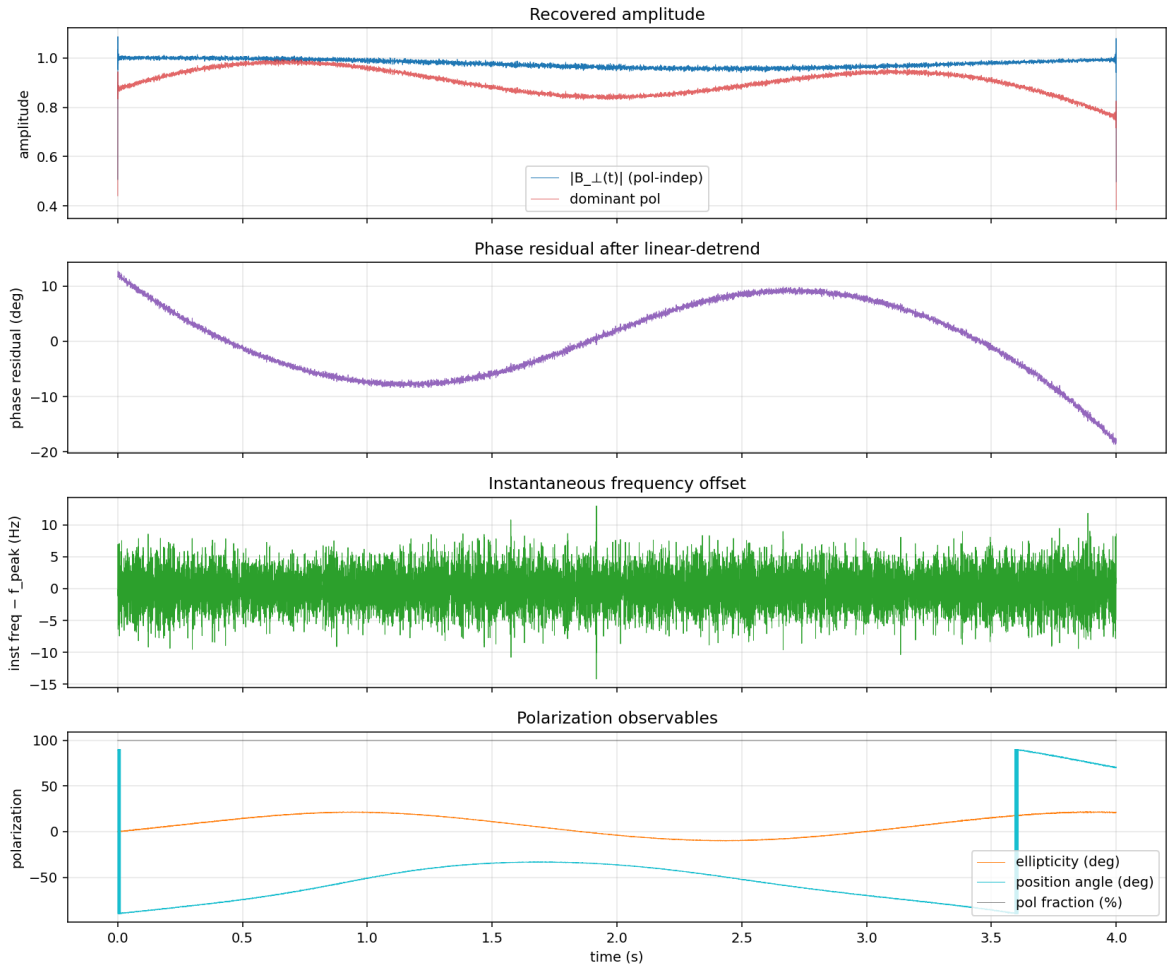


Figure 5: Diagnostics for the linear-pol capture. Top: the polarization-independent intensity $|\mathbf{B}_\perp(t)|$ is constant in time (Faraday rotation does not change wave intensity), but the dominant-pol amplitude shows the characteristic Faraday fading. Second: residual phase after linear detrending — bounded and small. Third: instantaneous frequency offset hovers near zero. Fourth: the position angle sweeps cleanly at $30^\circ/\mathrm{s}$ while ellipticity stays near 0° and polarization fraction stays at $100\,\%$.

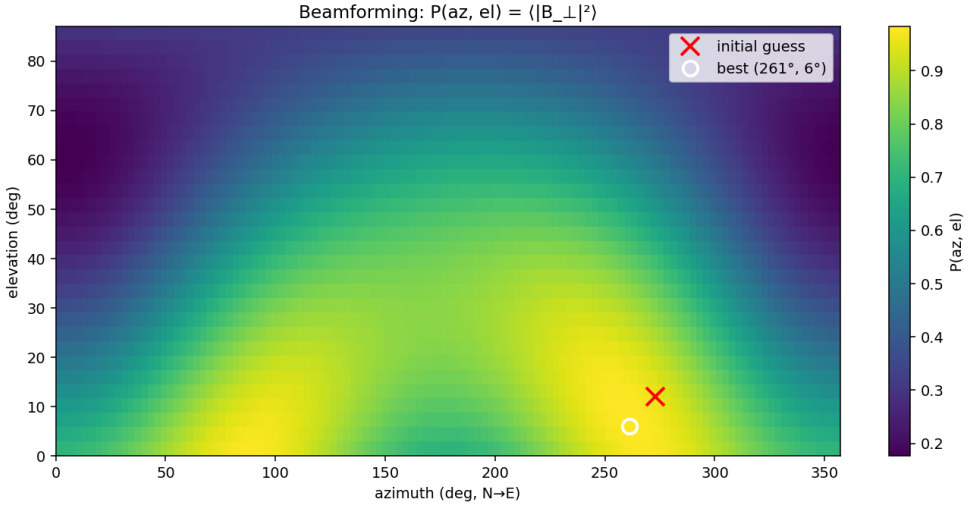


Figure 6: Beamforming sweep $P(\varphi, \theta) = \langle |\mathbf{B}_\perp|^2 \rangle$ for the linear-pol capture. The red `x` marks the initial guess (273°, 12°); the white circle marks the best-fit direction. Note the $\varphi \to \varphi + 180°$ ambiguity inherent to a single-point B-vector array.

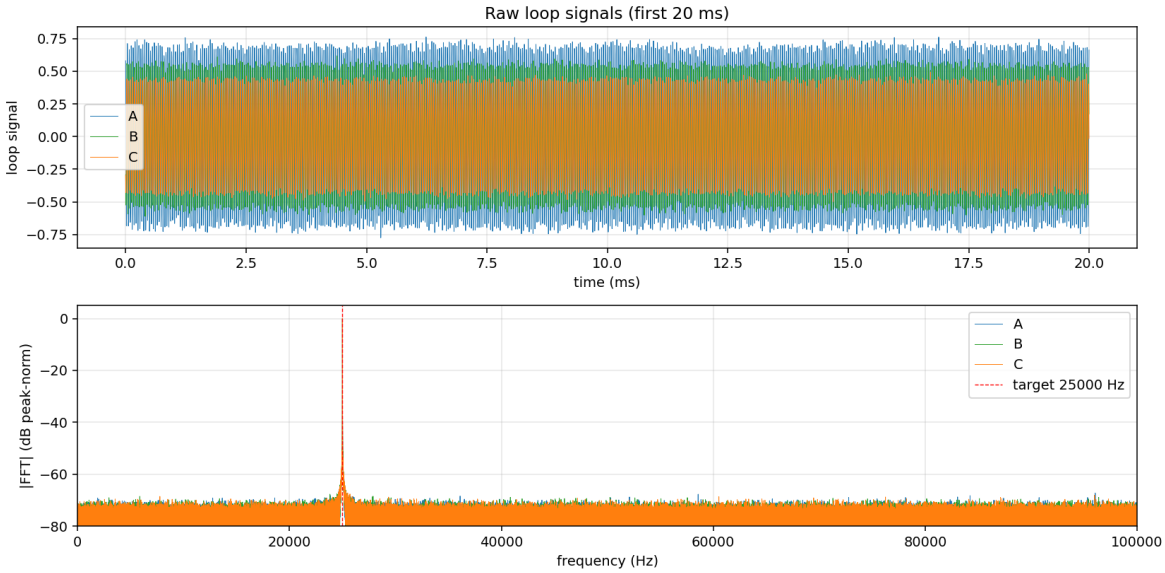


Figure 7: Raw signals and spectra for the RCP capture. All three loops see the same carrier with comparable amplitudes (the RCP wave's projection onto each loop is constant in magnitude up to a phase).

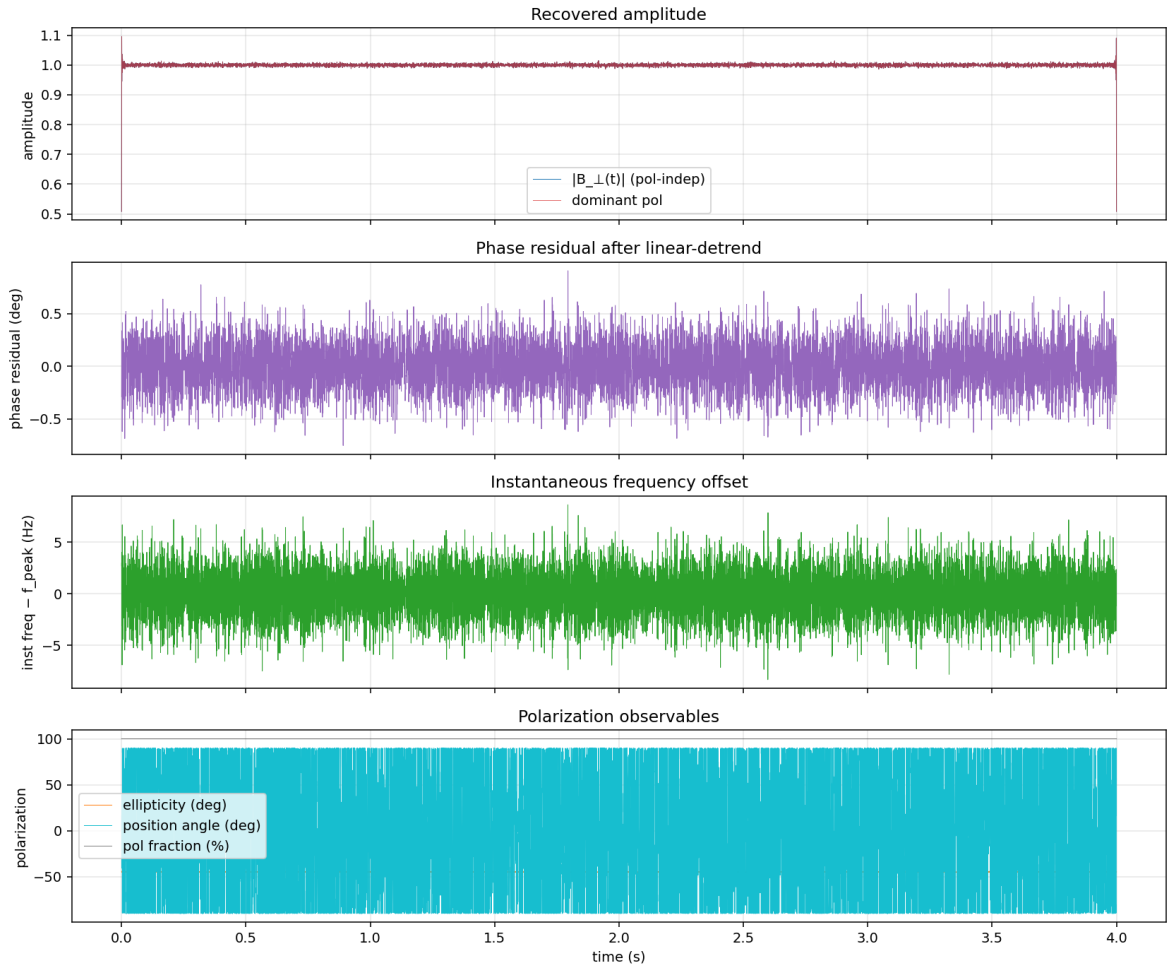


Figure 8: Diagnostics for the RCP capture. Both the polarization-independent intensity *and* the dominant-pol amplitude are flat: Faraday rotation of a circularly polarized wave is invisible to a magnitude-based estimator. Ellipticity sits at $\sim -45°$ throughout. The position-angle estimator is meaningless for circular polarization (no preferred linear axis) and consequently noisy.

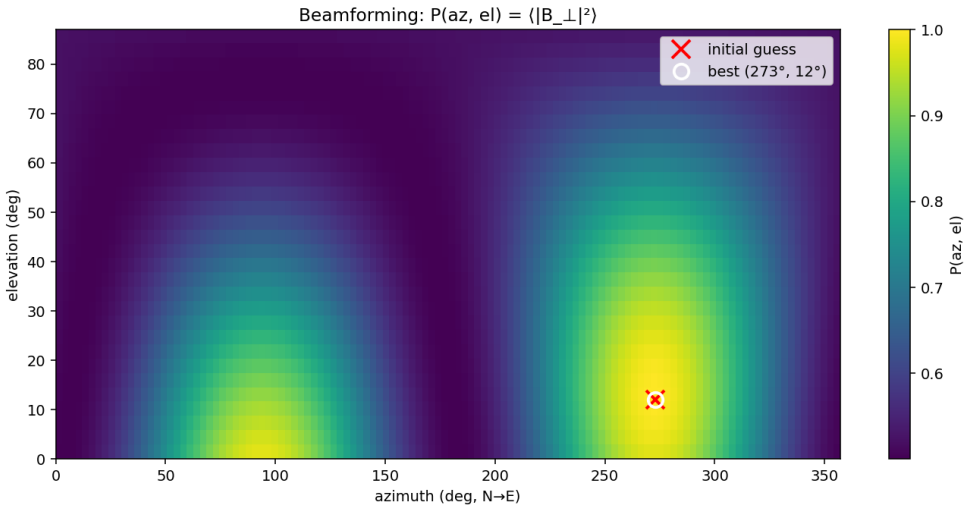


Figure 9: Beamforming sweep for the RCP capture. Direction recovery is unaffected by polarization state, as expected — the $|\mathbf{B}_\perp|^2$ estimator is intrinsically polarization-agnostic.